In [1]:
```
# a. Import necessary packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense

# b. Load training and testing data
train = pd.read_csv('sign_mnist_train.csv')
test = pd.read_csv('sign_mnist_test.csv')

# Separate labels and pixel values
X_train = train.drop('label', axis=1).values
y_train = train['label'].values

X_test = test.drop('label', axis=1).values
y_test = test['label'].values

# Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

# c. Define the network architecture
model = Sequential([
    Input(shape=(784,)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(25, activation='softmax')
])

# Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# d. Train the model
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1
)

# e. Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")
```

```
# f. Plot training loss and accuracy
plt.figure(figsize=(12, 5))

# Training loss
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Training accuracy
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()
```

```
Training data: (27455, 784)
Testing data: (7172, 784)
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
```

Model: "sequential"

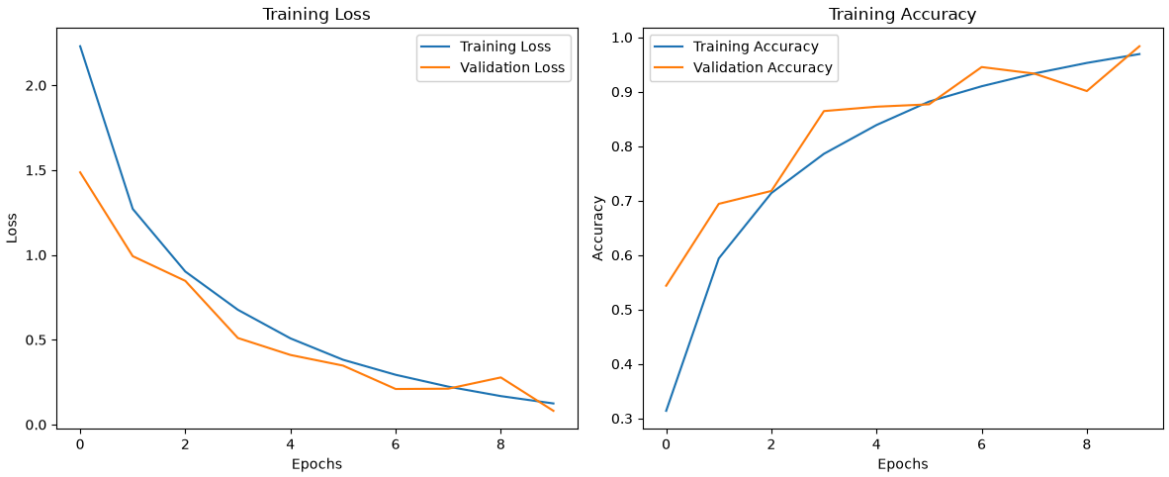
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 25)	1,625

Total params: 110,361 (431.10 KB)

Trainable params: 110,361 (431.10 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.3139 - loss: 2.2303 - val_
accuracy: 0.5437 - val_loss: 1.4867
Epoch 2/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.5938 - loss: 1.2713 - val_
accuracy: 0.6941 - val_loss: 0.9926
Epoch 3/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.7139 - loss: 0.9019 - val_
accuracy: 0.7178 - val_loss: 0.8470
Epoch 4/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.7860 - loss: 0.6765 - val_
accuracy: 0.8645 - val_loss: 0.5101
Epoch 5/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.8389 - loss: 0.5079 - val_
accuracy: 0.8725 - val_loss: 0.4102
Epoch 6/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.8819 - loss: 0.3819 - val_
accuracy: 0.8769 - val_loss: 0.3475
Epoch 7/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9101 - loss: 0.2934 - val_
accuracy: 0.9454 - val_loss: 0.2097
Epoch 8/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9335 - loss: 0.2235 - val_
accuracy: 0.9334 - val_loss: 0.2113
Epoch 9/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9530 - loss: 0.1678 - val_
accuracy: 0.9013 - val_loss: 0.2777
Epoch 10/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9692 - loss: 0.1244 - val_
accuracy: 0.9836 - val_loss: 0.0810
225/225 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7366 - loss: 1.0474
Test Loss: 1.047438383102417
Test Accuracy: 73.66145849227905 %
```




```
In [ ]:
```